Fine-Tuning and reinforcement Extensions for the Countdown Task

Joshua Rizo
jrizo@stanford.edu

June 10, 2025

1 One Page Extended Abstract

Motivation: The Countdown task presents a math reasoning problem, requiring models to form math equations using provided numbers to exactly match a specified target. In our project, we systematically addressed this task through the default project task consisting of supervised fine tuning, reinforcement learning using RLOO, and a texzt-time re-ranking extension.

Method: Initially, our supervised fine-tuning involved explicitly guiding the Qwen2.5-0.5B base model with annotated chains-of-thought. This allowed the model to internalize explicit numerical reasoning steps, significantly improving performance. Through hyperparameter tuning specifically adjusting learning rates, batch sizes, and implementing gradient accumulation we raised our initial subpar leaderboard performance from approximately 0.15 to around 0.288, nearing the desired baseline threshold. Subsequently, we employed reinforcement learning via the reinforce Leave one out, aiming to further refine model performance. Using optimized inference parameters (temperature = 0.6, top-p = 0.95), we saw moderate improvements to a leaderboard score of about 0.3169. While this was a positive increment, its minimal gain nature suggested potential dependence on inference parameters rather than RLOO's effectiveness, which we assume is due to the limited training time that we allotted for RLOO due to compute constraints. Given more computational resources, more training steps might be beneficial and highlight RLOO's advantages. Finally, motivated by recent works emphasizing efficient inference time computational strategies (Snell et al., 2024; Wang et al., 2023; Zhang et al., 2025), we introduced a test-time re-ranking extension.

Implementation and Results: This strategy involved generating multiple candidate solutions for each prompt at inference time, then selecting the best

response according to provided scoring mechanisms. Initially, responses were scored via the default `compute_score` function. However recognizing that many responses lacked conclusive answers indicated by the answer tags, we added an additional metric favoring responses containing explicit `<answer>` tags. Under constrained computational resources, we demonstrated the effectiveness of this approach, improving our leaderboard score significantly to **0.3295**.

Discussion and Conclusion: Notably, this extension required no additional training and leveraged additional inference time computation. While more demanding at inference time, this result highlights how practical modifications post training can substantially improve model performance. Our findings underscore the practical efficiency of inference time re-ranking. Future work will explore deeper reinforcement learning approaches and more sophisticated re-ranking strategies to further exploit model capabilities under computational constraints.

Abstract

We introduce an approach to improving model performance on the Countdown task as specified direclty in the Default Project Guidelines. Our work includes three stages: supervised fine-tuning of the Qwen2.5-0.5B model using chain of thought annotations to solve each nums-target prompt, reinforcement learning via RLOO inspired by Ouyang et al. (2022), and a test-time re-ranking extension to boost performance post-training. Specifically, our reranking extension generated multiple candidates per prompt at inference time, and then used methods of the provided scoring function for responses as well as scanning responses to see if an actual answer is present in order to rank responses and choose the best among these for the final answer. Our approach progressively increases accuracy from a supervised fine-tuning baseline to improvements through reinforcement learning and reranking. The final test-time reranking step leverages this multiple model generated candidate solutions to achieve substantial improvement without additional training.

2 Introduction

The Countdown task challenges language models to create arithmetic equations from a set of provided numbers to reach a specified target exactly. We address this by initially employing supervised fine tuning with annotated chains of thought to guide model reasoning explicitly. Subsequently, we utilize reinforce Leave one out to fine-tune the model further. Finally, we propose a test-time re-ranking strategy to effectively exploit multiple candidate solutions generated by the model at inference time, inspired by recent literature on test-time compute allocation and verifier methods (Snell et al., 2024; Wang et al., 2023; Zhang et al., 2025).

3 Task Introduction

The Countdown arithmetic task challenges a model to construct a valid arithmetic equation from a given set of numbers to exactly match a provided target number. This task evaluates a model's ability to reason numerically and systematically apply arithmetic operations (addition, subtraction, multiplication, and division) within strict constraints, such as using each number exactly once.

A typical Countdown prompt explicitly instructs the model on the format of the solution, emphasizing structured reasoning followed by a final solution wrapped around in special tags: `<think>` to delineate the reasoning process and `<answer>` for the final equation.

To illustrate, consider the following concrete example from our training dataset at (/Asap7772/cog-behav-all0strategies)

Example Prompt and Completion: User Prompt: "Using the numbers [95, 36, 32], create an equation that equals 91. You can use basic arithmetic

operations (+, -, *, /) and each number can only be used once. Show your work in <think> </think> tags. And return the final answer in <answer> </answer> tags, for example <answer> (1 + 2) / 3 </answer>."

Model Completion: "Let me solve this step by step. <think> First, let's identify the numbers closest to our target of 91. The number 95 is only 4 units away from 91. To achieve this, we can subtract the difference of the other two numbers, 36 and 32, since 36 minus 32 equals exactly 4. Therefore, the equation 95 - (36 - 32) precisely yields 91. </think>

<answer>95 - (36 - 32)</answer>"

This example clearly demonstrates the reasoning process the model must internalize and express. The structured prompt-completion format guides the model towards not only deriving solutions but also articulating the logical steps leading to the final answer

4 Related Work

Supervised fine-tuning guided by human feedback has demonstrated substantial effectiveness in improving model alignment on various tasks (Ouyang et al., 2022). Ouyang et al. notably demonstrated that reinforcement learning from human feedback (RLHF) enables models to generalize better to new prompts and adhere closely to desired outcomes.

Furthermore, Wang et al. (2023) introduced Self-Consistency, leveraging multiple sampled reasoning paths and selecting the majority consensus to boost accuracy significantly on reasoning tasks. Snell et al. (2024) further established that strategically allocating computational resources at inference can sometimes provide more effective gains than increasing model size alone. Most directly relevant to our extension, Zhang et al. (2025) introduced generative verifier methods that employ scoring mechanisms during inference to select the most accurate output from multiple model-generated candidates. This approach serves as the inspiration behind our re-ranking extension, leveraging additional computation at test-time effectively.

5 Method

5.1 Supervised Fine-Tuning

We prepare training samples by concatenating prompts with annotated chains of though and then asnwer, while also masking prompt tokens to force the model to predict reasoning and answer and only incur loss on those aspects:

$$\text{prompt end chain of thought end answer end}$$

Labels for prompt tokens are set to -100, focusing the loss solely on the think and answer segments.

5.2 RLOO

As defined in the project hand out, we we employ RLOO to reduce variance in policy gradient updates. It achieves this variance reduction through a baseline strategy, wherein the reward baseline for each sample is calculated as the average reward of the other samples generated from the current policy. We restate teh objective as written in the default project handout:

$$\frac{1}{k}\sum_{i=1}^{k}\left[R(y_{(i)},x)-\frac{1}{k-1}\sum_{j\neq i}R(y_{(j)},x)\right]\nabla\log\pi_\theta(y_{(i)}|x),\quad y_{(1)},\ldots,y_{(k)}\overset{i.i.d}{\sim}\pi_\theta(\cdot|x),$$

5.3 Test-Time Re-Ranking

Test-time re-ranking is motivated by the increasingly recognized strategy of allocating computational resources dynamically at inference time a concept extensively explored by Snell et al. (2024). They demonstrated that selectively allocating additional computational effort during inference can lead to performance improvements surpassing those obtained solely by scaling model size and or further training. This paradigm effectively leverages the computational budget to improve robustness and accuracy, which is particularly helpful and relevant to this default project since both time and compute resources were oscrace. Furthermore, the foundational work of Wang et al. (2023) on Self-Consistency has shown that sampling multiple reasoning pathways from a model and aggregating these outputs significantly improves the reasoning capabilities of language models. This underscores the benefits of capturing and leveraging the inherent variability in model predictions at inference time.

Building directly on these principles, Zhang et al. (2025) introduced "generative verifiers," sophisticated inference time mechanisms that evaluate and score generated candidates based on their predicted correctness. This verifier based approach directly informs our re-ranking strategy: we adopt a scoring-based evaluation at inference, where the the provided `compute-score` function for countdown task evaluates multiple candidate outputs. From here, we can take a simple `max` of the scores as a means of "choosing" the most optimal candidate, hopefully improving the performance of our model.

In practice, our re-ranking procedure involves the following steps:

Candidate Generation: For each Countdown prompt at inference, we sample multiple candidate equations from the trained model using VLLM sampling params (temperature = 0.6, top-p = 0.95). We are able to choose the m candidates that we want to sample, however, for the sake of time and computational tractability, we kept our m low to allow ample resource allocation. In our case specifically, we stuck with $m = 4$, meaning that we generate 4 candidates in total per prompt in the heldout set (leading to a total generation of 1,000 propmts $\times$ 4 candidates = 4,0000 generations.

Candidate Scoring: Each candidate is independently evaluated by the Countdown compute score function. This score functoin provides a robust "cor-

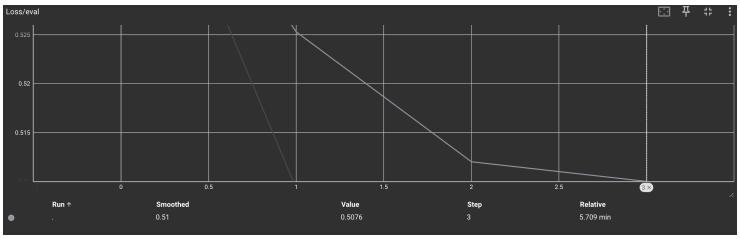
rectness" assessment, assigning higher scores to equations that correctly compute the target number and adhere to valid expressions. In the provided works, formulating this verifier is a significant task, so it's certainly convenient that, for our project, we have a defined way to evaluate the "goodness" of a response. Another way of scoring/evaluating corectness that we employed was by checking if a generated response gave a `answer` tag. We observed that many incorrect generated responses often did not converge in time to a conclusive answer, where often the response would run on past the 1024 token count and never be able to produce an answer, even if incorrect/correct. Thus, we also experimented with giving favorability to responses that had such tags in its generatino, via a simple regex. That way, we don't bother with responses that don't even have a viable answer, and this method is also less process dense than actually evaluating the task, since the entirety of its purview is maintained in a simple regex.

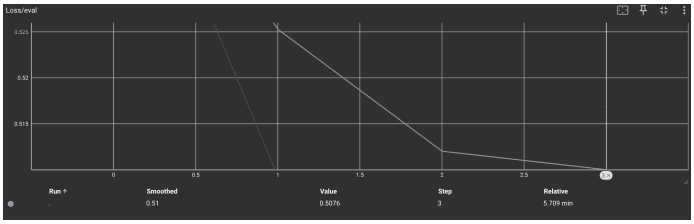
Final Selection: The candidate achieving the highest score is selected as the final output. This strategy directly exploits additional inference time computation to mitigate model uncertainty and effectively harness model diversity to achieve performance gains without further training overhead..

6 Experimental Setup

Our experimental process involved several iterative adjustments informed by preliminary results and external consultations. Initially, we experimented with supervised fine-tuning (SFT) configurations comprising 5 epochs at a learning rate of $1e-6$, a batch size of 2, and no gradient accumulation. This initial setup yielded unsatisfactory results, achieving a leaderboard score of approximately 0.15, substantially below the desired baseline of 0.3. Recognizing that we may not reach desirable results, we folloewd the advice of the edstem tips and implemented, implement gradient accumulation and adjust other hyperparameters.

ALSO, we upgraded our computing resources to an AWS g6e.xlarge instance (32 gb of memory) adopted a higher learning rate of $1e-5$ and adjusted training to 3 epochs, increasing the batch size to 4 and using gradient accumulation steps set to 7. By conducting further trials within very similar parameters to these, we successfully increased our model performance, consistently reaching leaderboard scores around 0.28, approaching the established target threshold. Following the establishment of our supervised fine-tuned base model at approximately a 0.288 leaderboard score, we proceeded to apply the RLOO to further model performance. We allowed RLOO training to run for 60 steps. When evaluating this fine-tuned model using inference parameters of temperature = 0.6 and top-p = 0.95, we observed an improved score of 0.3169 on the leaderboard, surpassing our baseline threshold. The observed improvement may have been more possibly attributable to the adjustments in VLLM inference parameters, as we tried different temperatures which notably caused variance in results rather than RLOO's intrinsic optimization effectiveness. Given more computing resources and time, further research could be conducted to rigorously verify the correctness and effectiveness of our RLOO implementation,





6.1 Test-Time Re-ranking Extension

We initially utilized the provided `compute-score` function designed explicitly for evaluating Countdown task prompts. However early observations indicated that many responses generated by the model did not include the necessary tags indicating the model frequently failed to converge upon a solution. To address this, we explicitly favoring responses that included a valid tag pair.

Due to limited compute resources, our evaluations were constrained to conducting inference tests twice on top of the SFT base model, with each evaluation limited to a candidate count of m= 4 to prevent exceeding resource limitations. These tests were conducted using temperature = 0.6 and top p = 0.95. Through this approach, we improved our leaderboard score from the original baseline of about 0.28 to an enhanced score of 0.3295. This demonstrated promising results, particularly since the extension operates solely during inference, requiring no additional resource-intensive training. The simplicity and computational efficiency of this method highlight its potential as a viable strategy for further performance enhancements without incurring substantial computational costs.

7 Result And Discussion

Our experimental outcomes are summarized based on leaderboard scores as the primary metric of success. Initially, the supervised fine-tuning without gradient accumulation and with suboptimal hyperparameters yielded a leaderboard score of approximately 0.15, significantly below our goal. After implementing recommended adjustments, including gradient accumulation and refined hyperparameters, our supervised fine-tuned base model performance improved notably to approximately 0.288.

The application of the RLOO estimator subsequently advanced the leaderboard score modestly to 0.3169, reflecting a beneficial yet limited enhancement.

This improvement, while meaningful, requires further verification to conclusively attribute success to RLOO. Most notably, our test-time re-ranking extension further boosted our model's performance to a leaderboard score of 0.3295. This extension provided substantial value by strategically leveraging inf time computations, demonstrating that significant performance gains can be efficiently achieved post-training without further costly retraining.

Overall, these results underscore the effectiveness of refining training hyperparameters, and the considerable advantages of strategic inference time enhancements in improving model performance.

8 Conclusion

Our project establishes the requisite supervised fine-tuning on Countdown, RLOO, and test-time re-ranking, significantly improving the Countdown task performance. Future work would explore further optimizations, more thorough RLOO training (more steps), and more fleshed out re ranking strategies, such as using more candidates and coupling different verifier methods (though, the given compute score does seem fitting from our judgement)

9 Team Contributions

Joshua Rizo - Sole contributor

References

[1] Long Ouyang et al., *Training language models to follow instructions with human feedback*, arXiv:2203.02155 (2022).

[2] Xuezhi Wang et al., *Self-Consistency Improves Chain of Thought Reasoning in Language Models*, arXiv:2203.11171 (2023).

[3] Charlie Snell et al., *Scaling LLM Test-Time Compute Optimally Can Be More Effective Than Scaling Model Parameters*, arXiv:2408.03314 (2024).

[4] Lunjun Zhang et al., *Generative Verifiers: Reward Modeling as Next-Token Prediction*, arXiv:2408.15240 (2025).